

```
#Import necessary libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
import numpy as np
```

```
#Load the dataset
df=pd.read_csv('/content/spotify_dataset.csv')
```

```
# Fill missing values
df['track_name'].fillna('Unknown',inplace=True)
df['track_artist'].fillna('Unknown',inplace=True)
df['track_album_name'].fillna('Unknown',inplace=True)
```

/tmp/ipython-input-615591200.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are sett
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df

```
df['track_name'].fillna('Unknown',inplace=True)
/tmp/ipython-input-615591200.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained  
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are sett  
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df
```

```
df['track_artist'].fillna('Unknown',inplace=True)
/tmp/ipython-input-615591200.py:4: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained  
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are sett  
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df
```

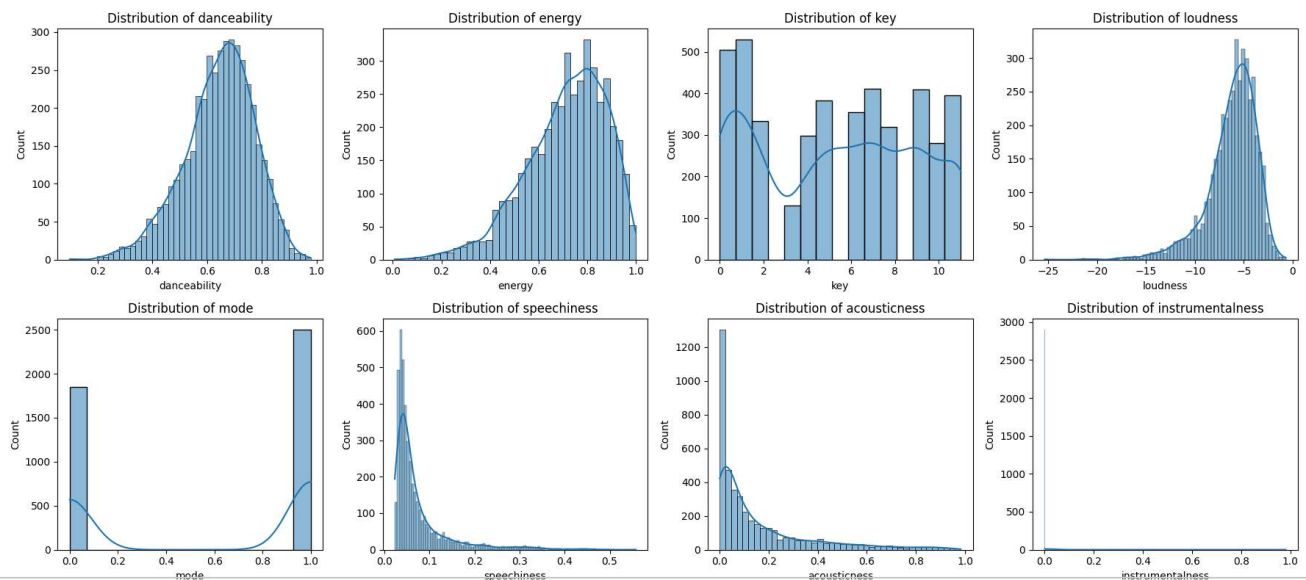
```
df['track_album_name'].fillna('Unknown',inplace=True)
```

```
df['track_album_release_date']=pd.to_datetime(df['track_album_release_date'],errors='coerce')
```

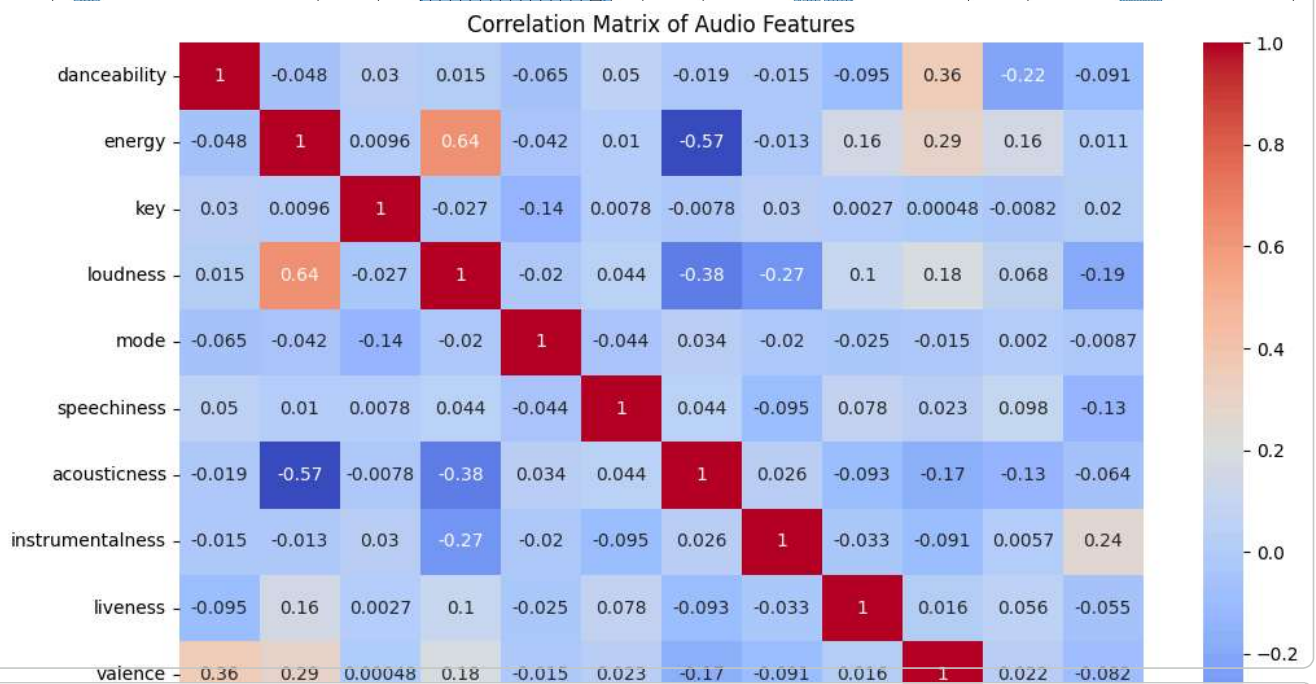
```
audio_features=['danceability','energy','key','loudness','mode','speechiness','acousticness','instrumentalness','liveness',  
                'valence','tempo','duration_ms']
```

```
X=df[audio_features]
scaler=StandardScaler()
X_scaled=scaler.fit_transform(X)
```

```
plt.figure(figsize=(18,12))
for i,col in enumerate(audio_features):
    plt.subplot(3,4,i+1)
    sns.histplot(df[col],kde=True)
    plt.title(f'Distribution of {col}')
plt.tight_layout()
plt.show()
```



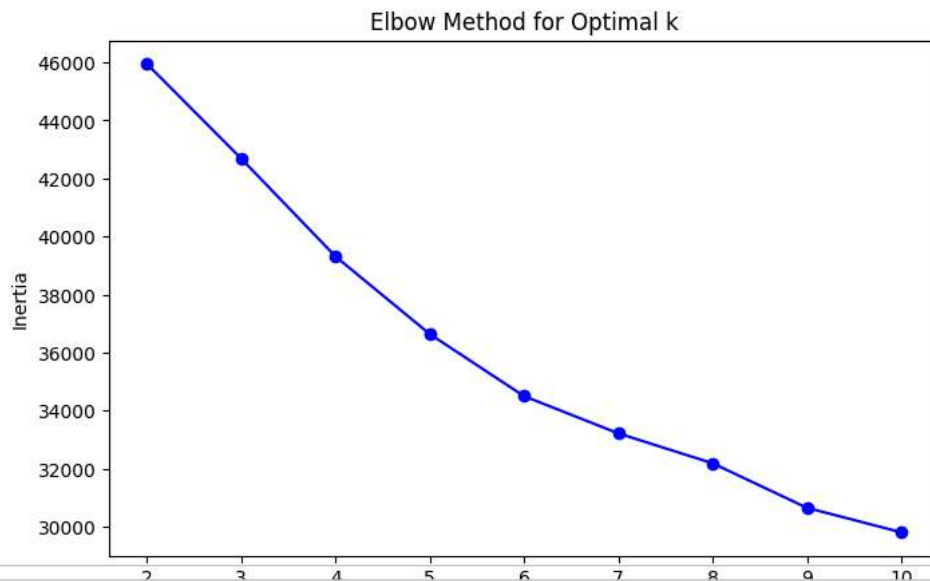
```
plt.figure(figsize=(12,8))
corr=df[audio_features].corr()
sns.heatmap(corr,annot=True,cmap='coolwarm')
plt.title('Correlation Matrix of Audio Features')
plt.show()
```



```
pca=PCA(n_components=2)
X_pca=pca.fit_transform(X_scaled)
```

```
# Determine optimal k using Elbow method
inertia=[]
K_range=range(2,11)
for k in K_range:
    km=KMeans(n_clusters=k,random_state=42)
    km.fit(X_scaled)
    inertia.append(km.inertia_)
```

```
plt.figure(figsize=(8,5))
plt.plot(K_range,inertia,'bo-')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Inertia')
plt.title('Elbow Method for Optimal k')
plt.show()
```

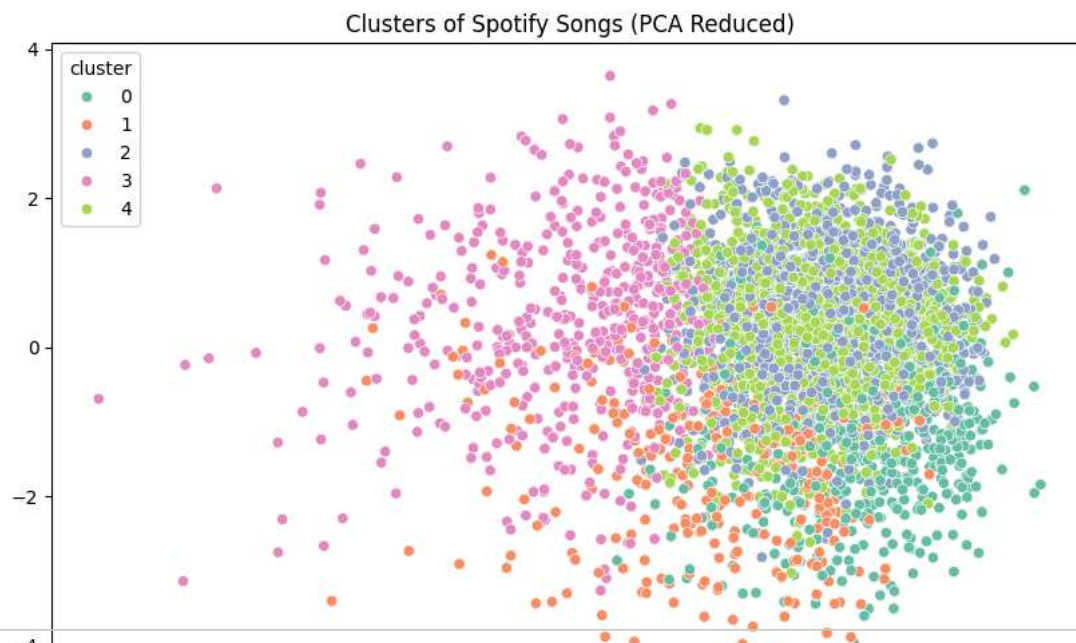


```
kmeans=KMeans(n_clusters=5,random_state=42)
df['cluster']=kmeans.fit_predict(X_scaled)
```

```
score=silhouette_score(X_scaled,df['cluster'])
print("Silhouette Score:",score)
```

Silhouette Score: 0.1141341049532245

```
plt.figure(figsize=(10,7))
sns.scatterplot(x=X_pca[:,0],y=X_pca[:,1],hue=df['cluster'],palette='Set2')
plt.title('Clusters of Spotify Songs (PCA Reduced)')
plt.show()
```



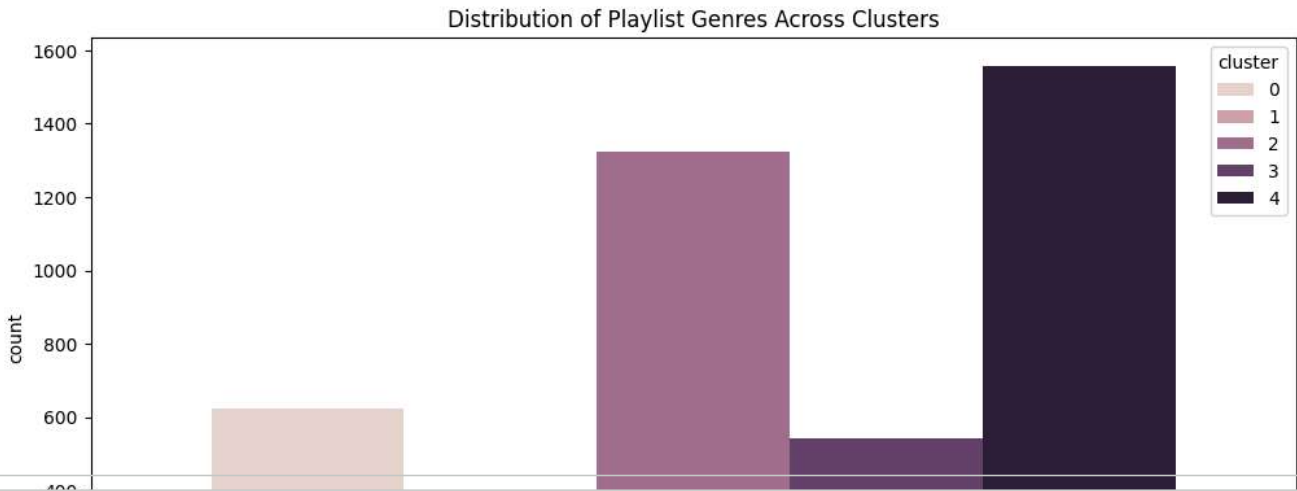
```
cluster_summary=df.groupby('cluster')[audio_features].mean()
print("\nCluster-wise Feature Means:")
print(cluster_summary)
```

cluster	danceability	energy	key	loudness	mode	speechiness
0	0.515377	0.783601	5.182692	-5.202728	0.660256	0.129178
1	0.646420	0.736437	5.793333	-8.231340	0.556667	0.052507
2	0.682497	0.750370	5.898868	-5.552391	0.000000	0.068649
3	0.608427	0.450130	5.223247	-9.527092	0.669742	0.077670
4	0.675474	0.742611	4.795761	-5.573988	1.000000	0.062796

	acousticness	instrumentalness	liveness	valence	tempo \
cluster					
0	0.086636	0.011934	0.245004	0.467958	156.028183
1	0.107165	0.667516	0.162895	0.470369	122.904843
2	0.109026	0.013367	0.175662	0.559200	116.524862
3	0.539678	0.054671	0.144852	0.365231	112.806923
4	0.103741	0.008899	0.170411	0.548696	114.410218

	duration_ms
cluster	
0	214576.980769
1	270859.776667
2	213574.737358
3	218233.190037
4	212125.050096

```
if 'playlist_genre' in df.columns:
    plt.figure(figsize=(12,6))
    sns.countplot(x='playlist_genre',hue='cluster',data=df)
    plt.title('Distribution of Playlist Genres Across Clusters')
    plt.xticks(rotation=45)
    plt.show()
```



```
def recommend_songs(track_name,n=5):
    song_data=df[df['track_name'].str.contains(track_name,case=False)]
    if song_data.empty:
        return f"No song found with name '{track_name}'"
    song_features=song_data.iloc[0][audio_features]
    song_scaled=scaler.transform([song_features])
    cluster_label=kmeans.predict(song_scaled)[0]
    cluster_songs=df[df['cluster']==cluster_label].copy()
    cluster_scaled=X_scaled[df['cluster']==cluster_label]
    distances=np.linalg.norm(cluster_scaled-song_scaled,axis=1)
    cluster_songs['distance']=distances
    recommendations=cluster_songs.nsmallest(n,'distance')[['track_name','track_artist','playlist_name']]
    return recommendations
print("\nRecommended Songs for 'Memories':")
print(recommend_songs('Memories'))
```

```
Recommended Songs for 'Memories':
   track_name  track_artist \
1  Memories - Dillon Francis Remix  Maroon 5
2426      Year 3000  Jonas Brothers
138  Friends (with BloodPop®)  Justin Bieber
3322      Secreto Perfecto  Jotdog
777  Come and Get Your Love - Radio Edit  Real McCoy

   playlist_name
1      Pop Remix
2426  post-teen pop
138      Dance Pop
3322  ELECTROPOP EN ESPAÑOL
777      90s Dance Hits

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but
warnings.warn(
```

